Technical Report: API Security Checklist for REST / Web APIs

Intern: Hansal Savaliya **Intern ID:** 2424 **Organisation:** RedKross Research Foundation Internship **Assignment:** API Security **Report date:** 24 October 2026 **GitHub repository:** `[Add repository URL before submission]` **Explanatory video:** `[Add video URL before submission]`

Project Overview

For my API Security assignment, I prepared a checklist for REST and web APIs, examples of safer request and response design, research notes, and a small Flask demo using fictional accounts and data. I organized the checklist around the security topics in my assignment.

The demo lets users sign in and manage only their own notes. It also checks request data, applies a simple in-memory rate limit, returns consistent errors and security headers, and shows deprecation headers on an older API route. Flask serves the browser interface and API from the same local server. On 24 October 2026, all 24 automated tests passed in 2.603 seconds using Python 3.12.10. The demo is for learning and is not a production service or a penetration test.

1. Objective and Scope

My assigned task was to develop a defensive checklist for REST or web APIs. It needed to cover authentication, object-level authorization, input validation, rate limiting, error handling, transport security, secrets, logging, and version/deprecation management. I also needed to provide a safe request/response example and references. The examples had to be local or fictional.

I made a local Flask demo to illustrate some of the checklist items. I did not test any third-party system. The accounts and notes are fictional, and the server binds to `127.0.0.1`.

2. Background Research

I reviewed the OWASP API Security Top 10 (2023), OWASP Cheat Sheets, NIST guidance, and relevant IETF RFCs. These sources helped me organize the checklist and explain risks such as broken object-level authorization, weak authentication, excessive resource use, security misconfiguration, and incomplete API inventories [1]. NIST SP 800-228 discusses API protection across development and operation [2]. I used relevant RFCs when writing about bearer tokens, rate limits, errors, deprecation, HTTP, and TLS [3–10].

I included a few publicly reported incidents to explain why these controls matter. They are background examples only: I did not investigate or reproduce them, and I have identified allegations as allegations in the research notes.

How I approached the research

1. Read the guidance and standards listed in References.
2. Grouped useful controls under the nine topics in my assignment.
3. Added priorities and ways to check the controls, with demo references where relevant.
4. Wrote fictional API examples.
5. Built and tested a local demo for a selection of the controls.

3. Deliverables

Deliverable	Location	Purpose
API security checklist	`checklist/api-security-checklist.md`	Prioritized and verifiable defensive review items
Safe request/response examples	`demo/safe-request-response-examples.md`	Fictional examples of expected API behavior
Flask demo and tests	`demo/app.py`, `demo/test_app.py`	Local implementation and automated checks
Browser interface	`demo/static/`	Local UI for login, dashboard, and note operations
Research notes	`report/research-notes.md`	Source summaries, context, and research caveats
References	`references.md`	Standards, guidance, incident reporting, and framework documentation
Test record	`screenshots/test-results.txt`	Environment, command, and latest full test-suite result
Walkthrough capture	`screenshots/demo_run_output.txt`	Captured output from an earlier local demo walkthrough

4. Checklist Design

The checklist has nine topic sections and a pre-release sign-off section. For each item, I included a priority, a suggested way to verify it, and a reference to the demo when applicable. P1/P2/P3 are priority labels used for this project. I also mapped relevant items to the OWASP API Security Top 10.

Topic	Checks
Authentication	12
Authorization	11
Input validation	11

Topic	Checks
Rate limiting and resource consumption	7
Error handling	7
Transport security	9
Secrets management	8
Logging and monitoring	8
Versioning, deprecation, and inventory	9

The checklist is meant to support reviews. It is not a certification or a replacement for testing the API being reviewed.

5. Demo Design and Implementation

5.1 Architecture and tools

I built the demo as one Flask application. It serves the API and browser interface from the same process, so no separate frontend server is needed. It uses Werkzeug password-hashing helpers, ItsDangerous signed timed tokens, and Python's standard `unittest` library.

Verified environment for the test run:

Component	Version / use
Python	3.12.10
Flask	3.1.3
Werkzeug	3.1.9
ItsDangerous	2.2.0
unittest	Python standard library
HTML, CSS, JavaScript	Browser UI served by Flask

5.2 Implemented controls

Control area	Demo behavior
Authentication	`POST /api/v1/login` issues a signed token with a 15-minute maximum age. Protected profile and note routes require a bearer token.
Password handling	Fictional account passwords are stored as salted password hashes. A dummy hash check is performed for unknown usernames to reduce username-dependent timing differences.

Control area	Demo behavior
Object-level authorization	Note reads, updates, and deletes check the owner against the token identity. A foreign or missing note receives the same 404 response.
Property-level authorization	Note responses expose only an explicit set of public fields. Create/update payloads allow only `title` and `body`.
Input validation	JSON content type is required for non-empty write requests; the body size is capped; note fields are type/length checked; unknown fields are rejected; malformed UUIDs return 404.
Pagination	Note listing accepts a positive `limit` capped at 50 and a non-negative `offset`; it returns page and total-count metadata.
Rate limiting	An in-memory sliding window applies 60 requests per minute to general traffic and 5 per minute to login traffic. Rate-limited responses include `Retry-After`; responses expose rate-limit metadata headers.
Error handling	Errors use a consistent JSON shape with a request ID. Unexpected exceptions are logged server-side and return a generic 500 response.
Response headers	Includes CSP, HSTS, `X-Content-Type-Options`, `X-Frame-Options`, `Referrer-Policy`, `Permissions-Policy`, and `Cache-Control: no-store`. HSTS does not provide TLS; deployment must provide HTTPS.
Secrets	`DEMO_SECRET_KEY` may be supplied through the environment; otherwise the process generates a random key at startup. No production credentials are included.
Logging	Request logs include a request ID, method, path, status, user ID when authenticated, and duration. Tokens, passwords, request bodies, and query strings are omitted.
Version lifecycle	`/api/v1` is current. The demonstration `/api/v0/notes` endpoint returns Deprecation, Sunset, and successor-version link headers.
Browser UI	`/` and `/ui` serve login, health, dashboard, and note-management screens. The bearer token is kept in memory; the theme preference is stored in browser local storage.

5.3 Running the demo

On Windows, from the `demo` directory, run `.\start_demo.bat`. It starts Flask in that terminal and opens the browser UI when the server responds. The browser UI is at `http://127.0.0.1:5000/`. Press Ctrl+C in the server terminal to stop it. The frontend is served by Flask and is not a separately launched process.

Fictional demo accounts are documented in the README. The app must remain local and must not be exposed as a public or production service.

6. Testing and Results

6.1 Automated test suite

I wrote 24 tests in `demo/test_app.py`. I ran them from the `demo` directory with:

```
.\.venv\Scripts\python.exe -m unittest -v test_app.py
```

It was run from the `demo` directory on 24 October 2026 using Python 3.12.10, Flask 3.1.3, Werkzeug 3.1.9, and ItsDangerous 2.2.0.

Result: 24 tests passed in 2.603 seconds (`OK`).

The tests cover:

- UI and static asset responses and the Content Security Policy.
- Health and authenticated profile endpoints.
- Missing/tampered authentication and equivalent login failure responses.
- Cross-user note reads, updates, deletes, and list scoping.
- Missing-versus-foreign note response equivalence.
- Payload validation, media type, body-size limits, malformed IDs, and public-field filtering.
- Login rate limiting, `Retry-After`, and rate-limit metadata headers.
- Generic error responses and hiding unexpected exception details.
- Security and deprecation headers.
- Logs not containing passwords or access tokens.

The suite verifies selected API behavior and UI route/asset delivery; it does not provide browser automation for interactive UI behavior and does not verify every checklist item. The result is recorded in `screenshots/test-results.txt`.

6.2 Manual walkthrough evidence

`screenshots/demo_run_output.txt` captures an earlier local run of `demo/run_demo.sh`. It demonstrates unauthenticated access, login, note creation, rejection of a mass-assignment field, denial of another user's note, owner access, version deprecation headers, and login rate limiting. Since the walkthrough logs in two users before submitting six invalid login attempts, those earlier successful logins consume part of the per-IP login allowance; therefore, the first three invalid attempts receive 401 and later attempts receive 429. This is a capture from that run, not a claim that the walkthrough was rerun with the latest test execution.

Older terminal screenshots show a prior 18-test run and are retained as historical evidence. The current automated result is the 24-test record in `screenshots/test-results.txt`.

6.3 Evaluation scope

All tests and demonstrations used this local application and fictional data. No production, third-party, or other external system was tested. These tests check selected behavior. They are not a penetration test or an independent security audit, and they do not prove that the application has no vulnerabilities.

7. Findings

While building the demo, I saw how important it is to check ownership on every operation involving a note. The tests also check that the server chooses note ownership, rejects unexpected fields, limits request sizes, returns generic errors, and handles both allowed and denied requests.

The demo also helped me separate code-level controls from operational ones. TLS, shared rate limiting, persistent secret storage, centralized logging, API inventories, and retiring old versions need additional deployment or organizational work. This small in-memory app does not implement those production controls.

8. Recommendations

If a team uses this checklist for a real service, I recommend that it:

1. Treat each checklist item as applicable, not applicable with a reason, or a tracked finding; require review of high-priority items before release.
2. Use a maintained schema or OpenAPI contract for request validation and contract testing.
3. Centralize or consistently reuse authorization policies, and test access across users and roles.
4. Put rate limits in a shared gateway or backing store when running multiple instances; tune thresholds to real workloads.
5. Terminate TLS with a supported, monitored configuration and verify HTTPS behavior in the deployment environment.
6. Use a managed secret store, separate keys by environment, and rehearse rotation and exposure response.
7. Maintain an endpoint/version inventory and retire deprecated APIs according to a communicated migration and sunset plan.
8. Add dependency and secret scanning to CI, protect logs, and alert on suspicious authentication, authorization, and rate-limit patterns.

9. Limitations

- Notes and rate-limit state are in memory and reset when the process restarts; this implementation is not suitable for multiple server instances.
- If `DEMO_SECRET_KEY` is unset, a fresh random signing key is created at startup, invalidating previously issued tokens after a restart.
- The app uses plain HTTP on localhost. TLS was not implemented or tested. The HSTS header alone does not encrypt traffic.

- The rate limiter is a teaching example, not a production-grade distributed limiter. It uses request IP information and does not provide a trusted proxy configuration.
- Token revocation, refresh tokens, MFA, roles, and administrative routes are not implemented.
- Logs are minimal application logs; centralized storage, alerting, retention, and a complete log-injection strategy are outside the demo scope.
- The UI's interactive behavior was not covered by browser-automation tests.
- The checklist and demo have not been independently reviewed or validated against a production API. GraphQL, WebSocket, and gRPC-specific guidance is outside the detailed scope.

10. Conclusion

The project provides the assigned API security checklist, safe request/response examples, references, research notes, and a fictional local API demo. The checklist is designed to be practical by pairing controls with suggested verification. The 24-test suite passed in the recorded environment and exercises selected authorization, validation, rate-limit, error, header, logging, and frontend-serving behavior. The limitations above define what the demo does not prove and what a production implementation would still need.

11. AI Assistance Declaration

Copilot in VS Code was used to assist with changes to the Flask demo and browser interface, project documentation, and execution of the automated tests. The intern should review the final source material, test evidence, and explanations, and should be prepared to explain the design and limitations in the required video. The intern remains responsible for the submitted work and for checking source references and claims.

12. References

The complete bibliography, including OWASP, NIST, IETF, incident-reporting, and framework sources, is maintained in `../references.md`. Key sources used in this report:

1. OWASP Foundation, *OWASP API Security Top 10 – 2023*. https://owasp.org/API-Security/editions/2023/en/0x11-t10/
2. NIST, *SP 800-228, Guidelines for API Protection for Cloud-Native Systems*, updated final version. https://csrc.nist.gov/pubs/sp/800/228/upd1/final
3. IETF, RFC 6750, *The OAuth 2.0 Authorization Framework: ****** Usage*. https://www.rfc-editor.org/rfc/rfc6750
4. IETF, RFC 6585, *Additional HTTP Status Codes* (429). https://www.rfc-editor.org/rfc/rfc6585
5. IETF, RFC 8594, *The Sunset HTTP Header Field*. https://www.rfc-editor.org/rfc/rfc8594
6. IETF, RFC 9745, *The Deprecation HTTP Response Header Field*. https://www.rfc-editor.org/rfc/rfc9745
7. IETF, RFC 8446, *The Transport Layer Security (TLS) Protocol Version 1.3*. https://www.rfc-editor.org/rfc/rfc8446

8. OWASP Cheat Sheet Series, *REST Security Cheat Sheet*.
https://cheatsheetseries.owasp.org/cheatsheets/REST_Security_Cheat_Sheet.html